Program Evaluation – Currency Conversion Program

1. How well the solution meets the requirements of the scenario

The program successfully meets all the requirements of the travel agent currency conversion scenario. It allows a user to input an amount in GBP and select from five currencies (USD, EUR, BRL, JPY, TRY). It performs conversion using fixed exchange rates stored in a dictionary for easy management. The program applies the correct transaction fee based on amount ranges, includes a staff discount of 5% when applicable, and handles input validation rejecting invalid or out-of-range amounts, invalid currencies, and excessive transaction values above £2500. Finally, it provides a clear transaction summary showing the converted amount, fee, discount, and total cost. Overall, the program runs smoothly, provides accurate calculations, and ensures user input is validated at every stage.

2. Quality and performance of the program

The program is of good quality in terms of structure, readability, and efficiency:

- Performance: The program executes quickly and performs all calculations instantly. Since it only handles simple arithmetic and string validation, it is highly efficient with negligible resource use.
- Error handling: All possible invalid user inputs (negative numbers, over-limit amounts, wrong currency codes) are handled gracefully with informative prompts.
- User experience: The interface is clear and text-based, making it easy to understand for users at a travel agency. Outputs are formatted neatly with labels and currency symbols for clarity.
- Maintainability: The use of functions (Convert_Currency, Transaction_Fee, and display_results) means the code can be easily modified if exchange rates or fee structures change in the future.

3. Choices made about coding conventions

Several consistent coding conventions were used to ensure readability and maintainability:

- Function names are descriptive and say what the function does (e.g., Convert_Currency, Transaction_Fee).
- Constants such as EXCHANGE_RATES are defined in uppercase to indicate that their values should not be changed.
- Variable names (e.g., GBP, Currency, fee, discount, total_cost) are descriptive and self-explanatory.
- Indentation and blank lines are used consistently to separate logical sections of code.
- Comments are included to explain the purpose of functions and key code segments, aiding future maintenance.
- Input validation and decision structures (IF-ELSE chains) follow a clean, readable style that reflects the pseudocode and flowchart.

These conventions ensure the code is professional, easy to follow, and adheres to common Python programming standards.

4. Changes made during the development process

During development, several improvements and corrections were made:

- Transaction fee logic corrected: The original fee conditions used equality (==) instead of range comparisons (>= and <=). This was fixed to ensure accurate fee application across all ranges.
- Input validation refined: Added loops (while True) so users are repeatedly prompted until valid input is provided.
- Staff discount option simplified: User responses were made case-insensitive (“yes”, “y”, “Y”, etc.) to improve usability.
- Output formatting enhanced: Added formatted output for currency amounts (e.g., two decimal places) and a clearly labelled summary section.
- Error handling for exceeding limit: Added logic to print an error and re-prompt if the amount exceeds £2500, as per scenario rules.

Each of these refinements improved program accuracy, user experience, and compliance with the task requirements.

Overall Evaluation

The final program fully meets the functional requirements of the travel agent currency conversion system. It is efficient, well-structured, readable, and thoroughly tested. All key features work as expected, with validation and error handling ensuring robust user interaction. Minor adjustments during development led to a final version that is accurate, user-friendly, and maintainable.